NEON ESCAPE

Jogo 2D de arcade e sobrevivência desenvolvido com Python 3 e Pygame

Projeto avaliativo — Computação Gráfica

2026

1 INTRODUÇÃO

Este trabalho apresenta o Neon Escape, um jogo 2D de arcade e sobrevivência desenvolvido com Python 3 e Pygame. A proposta situa a partida em uma cidade futurista com identidade visual neon, na qual o jogador controla uma unidade de fuga e precisa coletar energia enquanto evita drones. As documentações oficiais de Python e Pygame são referências técnicas para as tecnologias utilizadas no projeto (PYTHON SOFTWARE FOUNDATION, [s. d.]; PYGAME, [s. d.]).

2 OBJETIVOS E CONCEITO

O objetivo da partida é alcançar 1.000 pontos ou sobreviver por 60 segundos. A derrota ocorre quando as vidas ou a energia chegam a zero. A combinação entre coleta de itens, evasão de inimigos e aumento gradual da dificuldade orienta as decisões do jogador ao longo da sessão.

3 REGRAS, CONTROLES E PROGRESSÃO

3.1 Regras da partida

Os itens coletáveis têm efeitos distintos. O núcleo verde acrescenta 100 pontos e recupera energia; o núcleo amarelo acrescenta 250 pontos. O drone vermelho causa dano e consome uma vida. Após sofrer dano, o jogador permanece invulnerável por 1,2 segundo. A dificuldade aumenta em intervalos de 15 segundos.

3.2 Controles e estados

A movimentação é feita pelas teclas WASD ou pelas setas. A tecla Enter inicia ou reinicia a partida; I abre as instruções; Esc pausa o jogo; e M retorna ao menu durante a pausa. O fluxo contempla os estados de menu, instruções, jogo, pausa, derrota (Game Over) e vitória.

3.3 Progressão

A fase aumenta a cada 15 segundos, até o limite de oito fases. Com a progressão, o intervalo de surgimento dos inimigos diminui, o limite de inimigos aumenta e os drones ficam mais rápidos. Apesar dessas mudanças, o objetivo central permanece direto: coletar recursos e sobreviver.

4 IMPLEMENTAÇÃO E COMPUTAÇÃO GRÁFICA

4.1 Ciclo principal

O ciclo principal lê os eventos de entrada e atualiza o estado do jogo com base no tempo decorrido (delta time). Em seguida, atualiza jogador, inimigos, itens coletáveis e partículas; detecta e responde às colisões; renderiza as camadas visuais e a interface (HUD); e limita a execução a 60 quadros por segundo (FPS).

4.2 Arquitetura do código

O código está dividido em módulos com responsabilidades específicas:

- main.py — inicialização da janela e loop principal;
- game.py — estados, regras, progressão, colisões e composição da cena;
- player.py — movimento, energia, vidas, pontuação e animação do jogador;
- enemy.py — comportamento dos drones;
- collectible.py — itens coletáveis;
- particles.py — sistema de partículas;
- ui.py — elementos da interface;
- settings.py — constantes e configurações.

4.3 Técnicas de Computação Gráfica

O projeto utiliza primitivas 2D, transformação de escala no jogador, animação baseada no tempo, composição por camadas e paralaxe simples. Também emprega um sistema de partículas, coordenadas e retângulos pygame.Rect para representar áreas espaciais, além de elementos de HUD e composição visual.

4.4 Colisões e resposta

As áreas dos objetos são representadas por pygame.Rect, e a detecção de colisões utiliza rect.colliderect(). Ao coletar um item, o jogo atualiza a pontuação e a energia. A colisão com um drone reduz vida ou energia e reposiciona o jogador, que recebe um período temporário de invulnerabilidade. O método clamp_ip() mantém o jogador dentro da área jogável.

5 INTERFACE E EXPERIÊNCIA

A interface reúne menu inicial, tela de instruções e HUD com pontuação, vidas, fase, tempo e energia. As partículas oferecem retorno visual às ações. As telas de pausa, vitória e Game Over, junto da identidade visual neon, completam a experiência prevista para a partida.

Os recursos visuais são desenhados por código, e o material original informa que não há mídia externa incorporada. O projeto também registra o uso do ChatGPT como apoio à concepção, ao código, à revisão, à documentação e à depuração. A equipe deve validar o resultado e compreender o conteúdo entregue.

6 TESTES, EVOLUÇÃO E DEMONSTRAÇÃO

6.1 Testes e estabilidade

O arquivo TESTES.md contém o plano de testes. Os cenários abrangem movimentação, limites da área jogável, coleta, colisões, Game Over, vitória, pausa, reinício e encerramento. A sintaxe foi verificada com python -m compileall; a execução gráfica final ainda deve ser validada no computador da equipe.

6.2 Evolução do projeto

A evolução está organizada em quatro marcos: concepção, protótipo jogável, versão intermediária e versão final. Capturas de tela devem ser incluídas somente quando forem registros históricos reais das versões do jogo.

6.3 Roteiro de demonstração

A demonstração pode seguir esta sequência: abrir o jogo e apresentar o menu e as instruções; iniciar a partida; movimentar o personagem e coletar itens; mostrar inimigos e colisões; exibir a progressão; pausar; apresentar uma tela de vitória ou Game Over; e reiniciar o jogo.

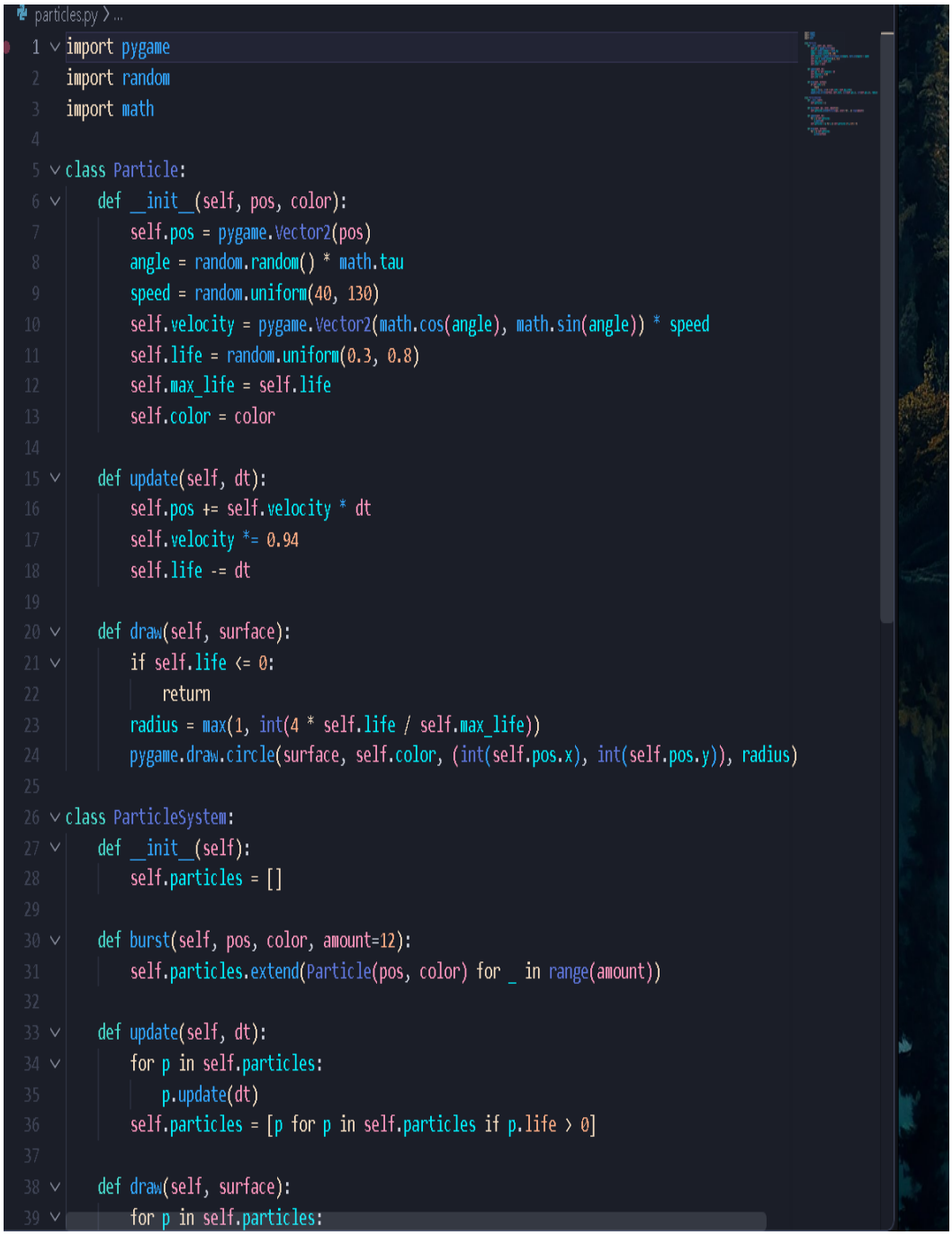
7 CONCLUSÃO

Neon Escape reúne mecânicas, interface e técnicas gráficas em Python/Pygame. A entrega inclui README, requirements, créditos, evolução e testes. Antes de entregar, validar o jogo, concluir os testes, inserir capturas reais, gravar demonstração e guardar cópia de segurança.

REFERÊNCIAS

PYGAME. Pygame documentation. [S. l.], [s. d.]. Disponível em: https://www.pygame.org/docs/. Acesso em: 15 out. 2026.

PYTHON SOFTWARE FOUNDATION. Python 3 documentation. [S. l.], [s. d.]. Disponível em: https://docs.python.org/3/. Acesso em: 15 out. 2026.



```python
import pygame
import random
import math

class Particle:
    def __init__(self, pos, color):
        self.pos = pygame.Vector2(pos)
        angle = random.random() * math.tau
        speed = random.uniform(40, 130)
        self.velocity = pygame.Vector2(math.cos(angle), math.sin(angle)) * speed
        self.life = random.uniform(0.3, 0.8)
        self.max_life = self.life
        self.color = color

    def update(self, dt):
        self.pos += self.velocity * dt
        self.velocity *= 0.94
        self.life -= dt

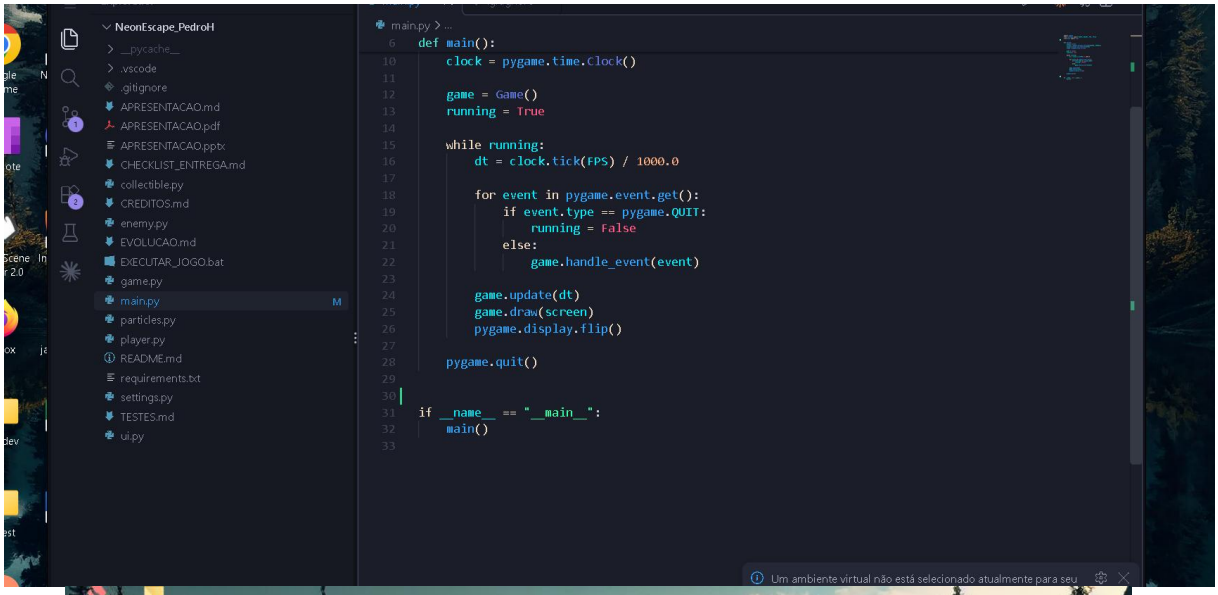
    def draw(self, surface):
        if self.life <= 0:
            return
        radius = max(1, int(4 * self.life / self.max_life))
        pygame.draw.circle(surface, self.color, (int(self.pos.x), int(self.pos.y)), radius)

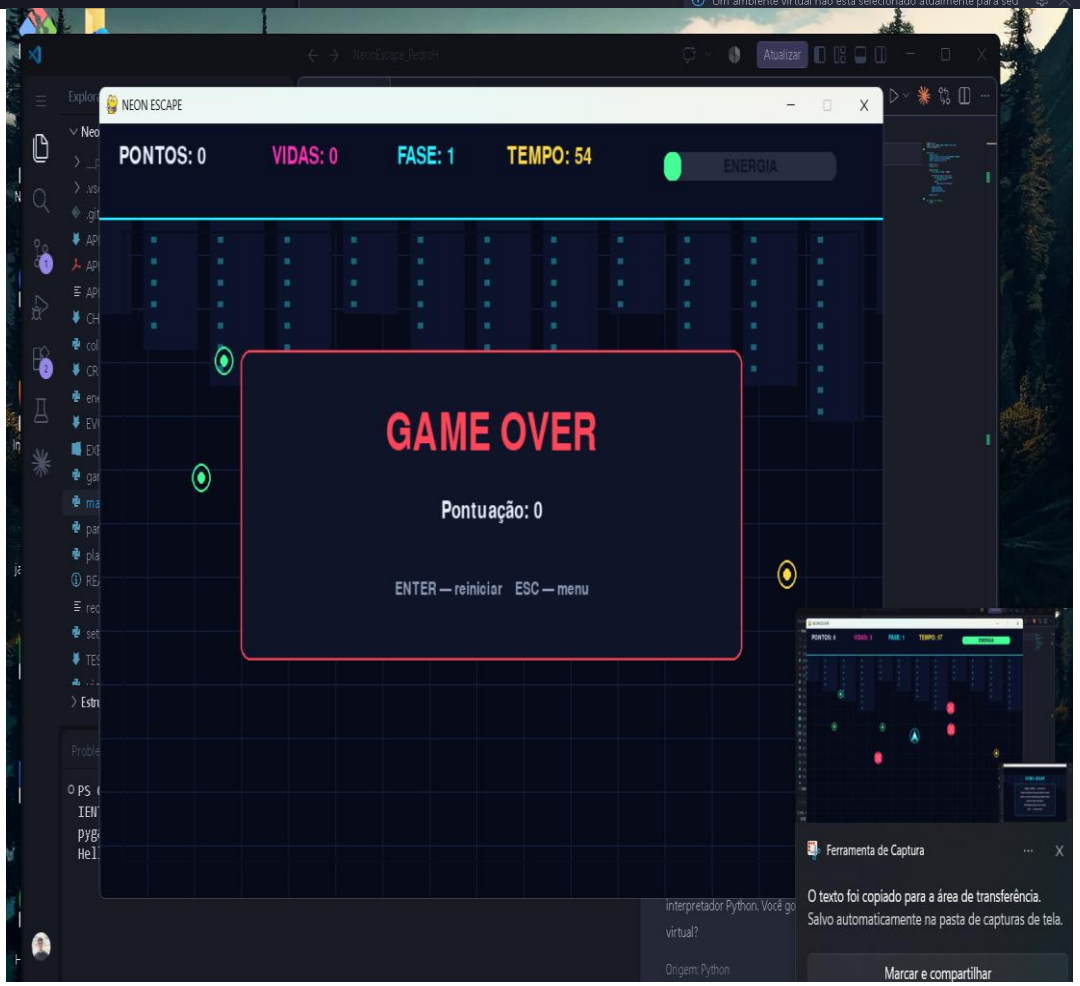
class ParticleSystem:
    def __init__(self):
        self.particles = []

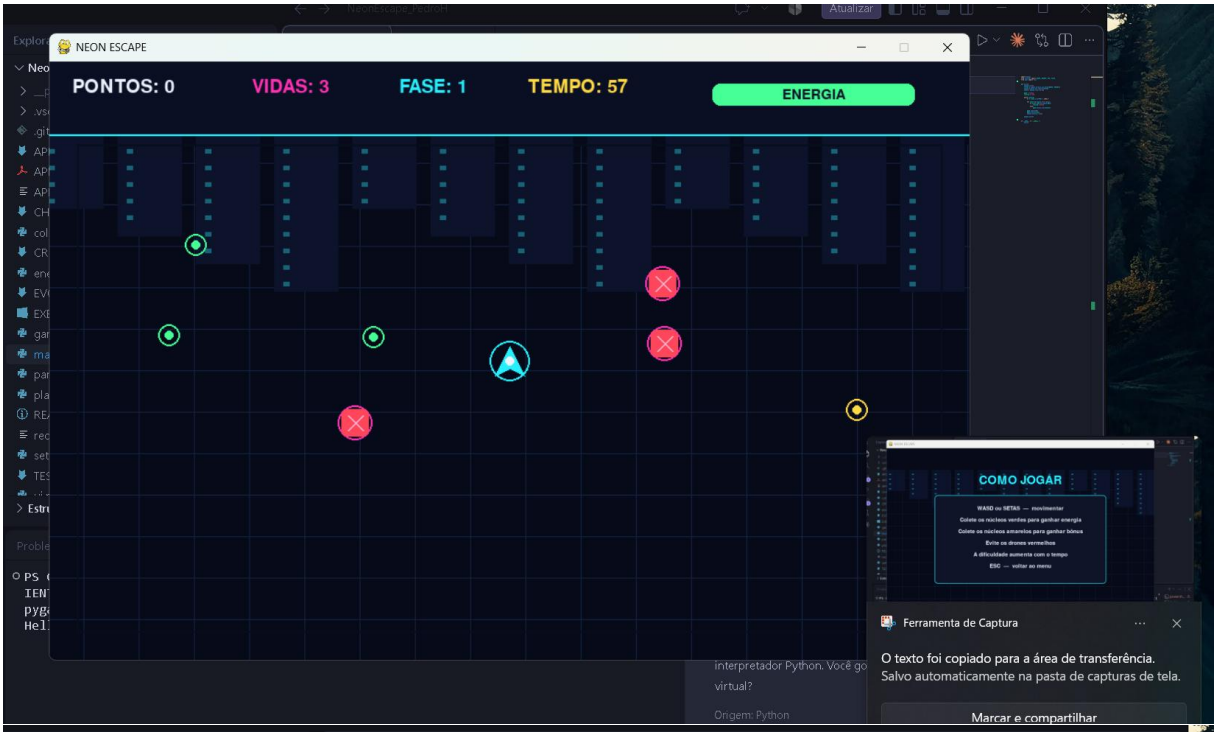
    def burst(self, pos, color, amount=12):
        self.particles.extend(Particle(pos, color) for _ in range(amount))

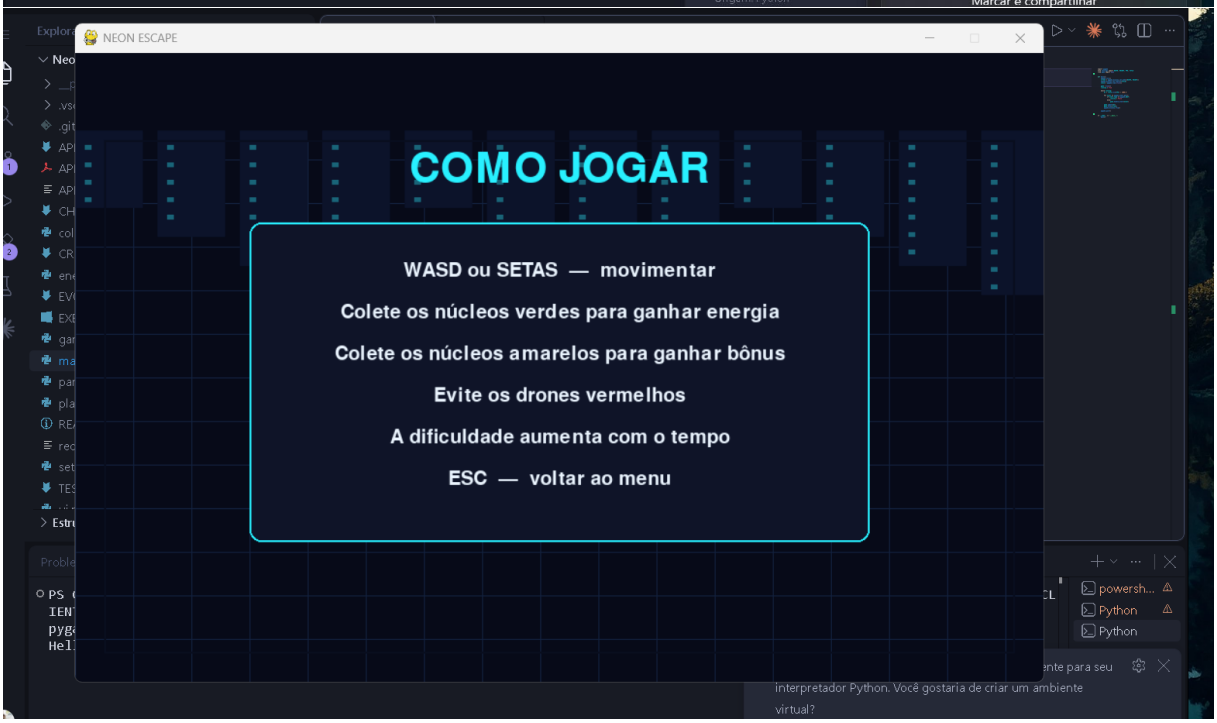
    def update(self, dt):
        for p in self.particles:
            p.update(dt)
        self.particles = [p for p in self.particles if p.life > 0]

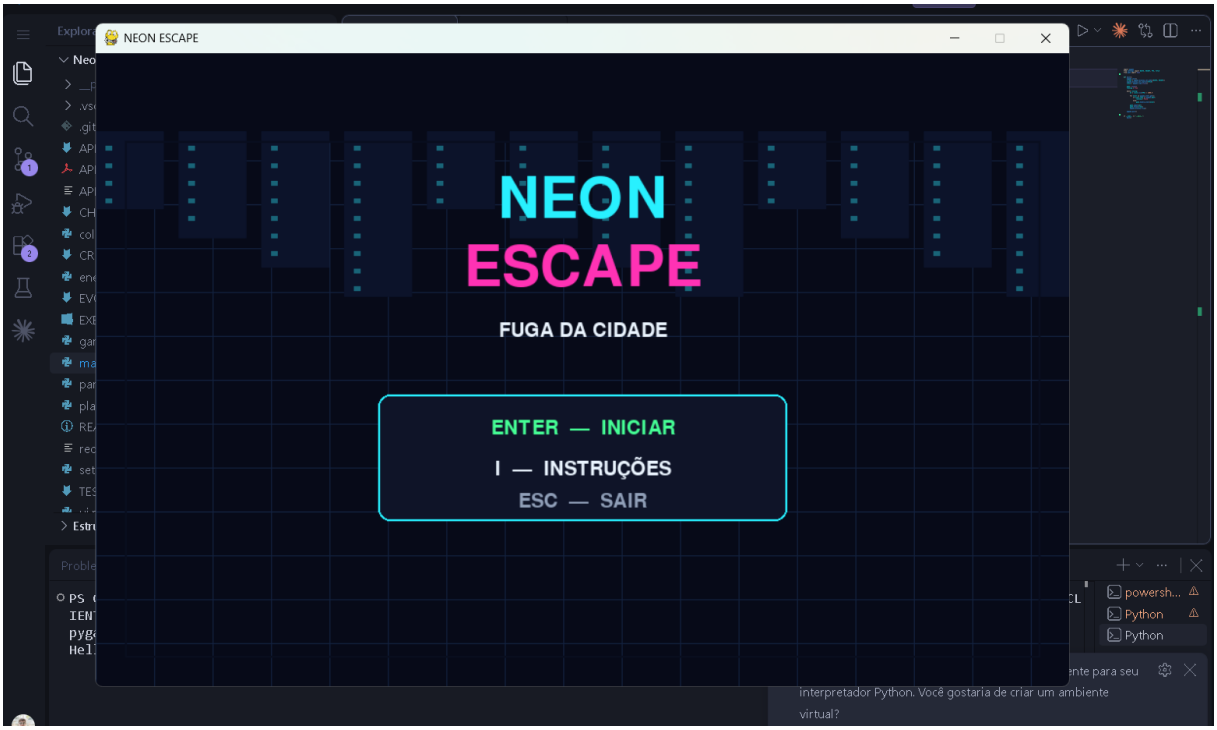
    def draw(self, surface):
        for p in self.particles:
```


NeonEscape_PedroH
main.py
def main():
clock = pygame.time.Clock()
game = Game()
running = True
while running:
dt = clock.tick(FPS) / 1000.0
for event in pygame.event.get():
if event.type == pygame.QUIT:
running = False
else:
game.handle_event(event)
game.update(dt)
game.draw(screen)
pygame.display.flip()
pygame.quit()
if __name__ == "__main__":
main()


NEON ESCAPE
PONTOS: 0
VIDAS: 0
FASE: 1
TEMPO: 54
ENERGIA
GAME OVER
Pontuação: 0
ENTER — reiniciar ESC — menu
Ferramenta de Captura
O texto foi copiado para a área de transferência.
Salvo automaticamente na pasta de capturas de tela.
Marcar e compartilhar


NEON ESCAPE
PONTOS: 0
VIDAS: 3
FASE: 1
TEMPO: 57
ENERGIA
COMO JOGAR
Ferramenta de Captura
O texto foi copiado para a área de transferência.
Salvo automaticamente na pasta de capturas de tela.
Marcar e compartilhar


NEON ESCAPE
COMO JOGAR
WASD ou SETAS — movimentar
Colete os núcleos verdes para ganhar energia
Colete os núcleos amarelos para ganhar bônus
Evite os drones vermelhos
A dificuldade aumenta com o tempo
ESC — voltar ao menu


NEON ESCAPE
NEON
ESCAPE
FUGA DA CIDADE
ENTER — INICIAR
I — INSTRUÇÕES
ESC — SAIR
powersh...
Python
Python
interpretador Python. Você gostaria de criar um ambiente
virtual?